VECTOR

MICROSAR vMemAccM

Technical Reference

Version 4.00.00

Authors	Charlotte Ricke, Tomas Ondrovic, Sebastian Reischl
Status	Released

Document Information

History

Author	Date	Version	Remarks
Charlotte Ricke	2018-03-16	1.00.00	Initial creation
Tomas Ondrovic Charlotte Ricke	2018-04-09	2.00.00	Added critical section description (chapter 4.2). STORY-3380: changes due to support of hardware specific functions and multi instance vMem. Updated chapters 3.9.6, 3.10, 4.1.2, 5.1, 5.2.8 and 5.2.11. Added chapter 4.1.3 describing interfaces of vMem which are specified by vMemAccM.
Charlotte Ricke	2018-09-24	2.01.00	STORYC-4408: Support write and erase burst in vMemAccM. Adapted description of Length parameter in chapter 5.2.4 and 5.2.8. Replace “FOTA” with “Software Download”.
Tomas Ondrovic	2018-12-18	2.02.00	STORY-9128/ STORYC-7262: vMemAccM shall provide information about currently processed job. (added chapter 5.2.15, extended 5.1 and feature tables)
Tomas Ondrovic	2019-01-17	2.03.00	STORY-9427/ STORYC-7272: vMemAccM shall support address area variants. (added chapter 3.6.1 and 5.2.16, updated 5.1)
Tomas Ondrovic	2019-02-04	2.04.00	STORY-8665: vMemAccM shall retry a failed write job. STORY-9253: vMemAccM shall retry a failed erase job. (added chapter 3.10)
Charlotte Ricke	2019-02-28	2.05.00	STORY-9456: vMemAccM shall evaluate ECC error results returned by vMem.
Tomas Ondrovic	2019-07-03	2.06.00	STORY-12777: vMemAccM shall provide a service to get notified about ECC errors (chapter 3.5, 5.1, 5.2.17)
Sebastian Reischl	2019-07-26	3.00.00	MWDG-120: vMemAccM shall provide a service to check a specific NV RAM address space to be blank/ erased.
Tomas Ondrovic	2019-09-02	3.00.01	MWDG-1884: vMemAccM shall clarify the RAM alignment requirements (added chapter 3.11, adjusted others).
Tomas Ondrovic	2019-10-29	3.00.02	ESCAN00102322: vMemAccM_HwSpecificReq accepts data size 0 but does not accept data pointer ==

			NULL_PTR (chapter 3.9.6).
Tomas Ondrovic	2020-04-07	3.01.00	Reworked chapter 3.3. MWDG-2917: Implement the access synchronization in a multi core system (added 3.12)
Tomas Ondrovic	2020-05-14	4.00.00	MWDG-3139: vMemAccM shall cancel an ongoing job according to AUTOSAR (chapter 3.4.4) Improvements within chapter 3.12.2.1.

Reference Documents

No.	Source	Title	Version
[1]	AUTOSAR	AUTOSAR_SWS_DevelopmentErrorTracer.pdf	3.2.0
[2]	Vector	TechnicalReference_vMem.pdf	see delivery



Caution
We have configured the programs in accordance with your specifications in the questionnaire. Whereas the programs do support other configurations than the one specified in your questionnaire, Vector´s release of the programs delivered to your company is expressly restricted to the configuration you have specified in the questionnaire.

© 2020 Vector Informatik GmbH

Contents

1 Component History 9

2 Introduction 10
2.1 Architecture Overview 11
2.2 Component Interfaces 13

3 Functional Description 14
3.1 Features 14
3.2 Initialization 14
3.3 Main Functions 14
3.3.1 Main Function triggering 15
3.4 Job Results 15
3.4.1 VMEMACCM_JOB_PENDING 15
3.4.2 VMEMACCM_JOB_OK 15
3.4.3 VMEMACCM_READ_CORRECTED_ERRORS 16
3.4.4 VMEMACCM_JOB_CANCELED 16
3.4.5 VMEMACCM_JOB_FAILED 16
3.4.6 VMEMACCM_READ_UNCORRECTABLE_ERRORS 16
3.4.7 VMEMACCM_MEM_NOT_BLANK 16
3.5 Error Notification 16
3.6 Memory Address Mapping 17
3.6.1 Memory address mapping variants 19
3.7 Priority Based Job Processing 20
3.8 Burst Feature 20
3.9 Partitioning of Asynchronous Jobs 21
3.9.1 Read 21
3.9.2 Write 21
3.9.3 Erase 21
3.9.4 Compare 21
3.9.5 IsBlank 21
3.9.6 Hardware Specific Requests 21
3.10 Retries of a failed job 22
3.10.1 Retry a failed burst size request 23
3.10.2 Retry configuration 23
3.10.3 Supported vMemAccM requests 23
3.11 RAM Buffers 24
3.12 Multi partition synchronization 24
3.12.1 Configuration 25

3.12.2 Memory Mapping 26
3.13 Error Handling 29
3.13.1 Development Error Reporting 29

4 Integration 32
4.1 Scope of Delivery 32
4.1.1 Static Files 32
4.1.2 Dynamic Files 32
4.1.3 Interfaces of the vMem 32
4.2 Critical Sections 33

5 API Description 34
5.1 Type Definitions 34
5.2 Services provided by vMemAccM 37
5.2.1 vMemAccM_InitMemory 37
5.2.2 vMemAccM_Init 37
5.2.3 vMemAccM_Read 38
5.2.4 vMemAccM_Write 38
5.2.5 vMemAccM_Erase 39
5.2.6 vMemAccM_Compare 40
5.2.7 vMemAccM_IsBlank 40
5.2.8 vMemAccM_HwSpecificReq 41
5.2.9 vMemAccM_MainFunction 42
5.2.10 vMemAccM_Cancel 42
5.2.11 vMemAccM_GetJobResult 43
5.2.12 vMemAccM_GetMemoryInfo 43
5.2.13 vMemAccM_SetReadOnlyMode 44
5.2.14 vMemAccM_GetProcessedLength 44
5.2.15 vMemAccM_GetState 45
5.2.16 vMemAccM_SetActiveVariant 46
5.2.17 vMemAccM_ErrorNotification 46
5.2.18 vMemAccM_GetVersionInfo 47
5.3 Services used by vMemAccM 47
5.4 Callback Functions 48
5.5 Configurable Interfaces 48
5.5.1 Notifications 48
5.5.2 Callout Functions 48

6 Configuration 49
6.1 Configuration Variants 49
6.2 Configuration with DaVinci Configurator 49

6.2.1 Read Request Length 49
6.2.2 Memory Address Mapping 49

7 Glossary and Abbreviations 51
7.1 Glossary 51
7.2 Abbreviations 51

8 Contact 53

Illustrations

Figure 2-1 AUTOSAR 4.x Architecture Overview 11
Figure 2-2 Component Overview of one most likely use case 12
Figure 2-3 Interfaces to adjacent modules of the vMemAccM 13
Figure 3-1 Memory address mapping 19
Figure 3-2 Multi partition synchronization architecture example (2 cores) 25

Tables

Table 1-1 Component history 9
Table 3-1 Provided Features 14
Table 3-2 vMemAccM and vMem result combinatorics in case of Error Notification. 17
Table 3-3 Service IDs 30
Table 3-4 Errors reported to DET 30
Table 3-5 Development Error Reporting: Assignment of checks to services 31
Table 4-1 Static files 32
Table 4-2 Generated files 32
Table 5-1 Type definitions 35
Table 5-2 vMemAccM_MemoryInfoType 36
Table 5-3 vMemAccM_StateType 36
Table 5-4 vMemAccM_InitMemory 37
Table 5-5 vMemAccM_Init 37
Table 5-6 vMemAccM_Read 38
Table 5-7 vMemAccM_Write 39
Table 5-8 vMemAccM_Erase 39
Table 5-9 vMemAccM_Compare 40
Table 5-10 vMemAccM_IsBlank 41
Table 5-11 vMemAccM_HwSpecificReq 42
Table 5-12 vMemAccM_MainFunction 42
Table 5-13 vMemAccM_Cancel 42
Table 5-14 vMemAccM_GetJobResult 43
Table 5-15 vMemAccM_GetMemoryInfo 44
Table 5-16 vMemAccM_SetReadOnlyMode 44
Table 5-17 vMemAccM_GetProcessedLength 45
Table 5-18 vMemAccM_GetState 45
Table 5-19 vMemAccM_SetActiveVariant 46
Table 5-20 vMemAccM_ErrorNotification 47
Table 5-21 vMemAccM_GetVersionInfo 47
Table 5-22 Services used by the vMemAccM 48
Table 7-1 Glossary 51
Table 7-2 Abbreviations 52

1 Component History

The component history gives an overview over the important milestones that are supported in the different versions of the component.

Component Version	New Features
1.00.00	Creation of component
2.00.00	Added support of hardware specific functions and multi instance vMem. Affected services are HwSpecificReq and GetMemoryInfo.
2.01.00	Added support of write and erase burst.
2.02.00	Added GetState API.
2.03.00	Added Address Area Variants.
2.04.00	Added failed job retries.
2.05.00	Added evaluation of ECC error results.
2.06.00	Added ErrorNotification (in context of ECC error handling).
3.00.00	Added IsBlank API.
3.01.00	Added multi partition synchronization.
4.00.00	Reworked cancelation to match AUTOSAR standard.

Table 1-1 Component history

2 Introduction

This document describes the functionality, API and configuration of the MICROSAR BSW module vMemAccM.

Supported AUTOSAR Release:	4	
Supported Configuration Variants:	pre-compile	
Vendor ID:	VMEMACCM_VENDOR_ID	30 decimal (= Vector-Informatik, according to HIS)
Module ID:	VMEMACCM_MODULE_ID	255 decimal

The vMemAccM (Memory Access Manager) coordinates the NV (non volatile) memory access by multiple users in order to avoid conflicts with this shared resource.

The basic functionalities offered by the vMemAccM to access the memory are read, write, erase and compare. Note, that the vMemAccM does not access the memory directly itself, but merely forwards requests to the underlying vMem drivers (Memory Driver, for a documentation of the vMem see [2]).

The vMemAccM was originally created within the new Vector solution for Software Download, which is also its most likely use case. In this case the vMemAccM coordinates the memory access between the classical AUTOSAR memory stack and the Software Download stack. Another likely use case is the memory access by the HSM (Hardware Security Module).

To manage the memory access, the vMemAccM implements the following main features:

- To avoid clashes within the memory resource, e.g. the flash memory, the vMemAccM processes only one asynchronous memory access job at a time. However, to still be able to accept multiple jobs for different regions of memory the vMemAccM provides a queueing mechanism. Furthermore, the order in which the jobs within the queue are being processed is based on priorities to best meet the requirements of the various users.
- In order to offer higher layers independence from the actual memory distribution within the underlying vMem modules, the vMemAccM performs a memory address mapping. Upper layers only use virtual addresses to access memory. The virtual memory is structured in so called address areas. The vMemAccM performs the mapping from virtual addresses to pseudo-physical addresses and forwards requests to the underlying vMem modules.
- To enable higher system response time for higher job priority execution, the vMemAccM divides the asynchronous jobs up into jobs of smaller length, which are forwarded to the underlying vMem. This way the related vMem is only blocking for a short time, before a different asynchronous job with higher priority could be started. The length of the forwarded job depends on the specific request and configuration.

2.1 Architecture Overview

Note that vMemAccM is not an AUTOSAR module. However, if one tried to place it in the AUTOSAR architecture it would be a complex driver as shown in the following figure.

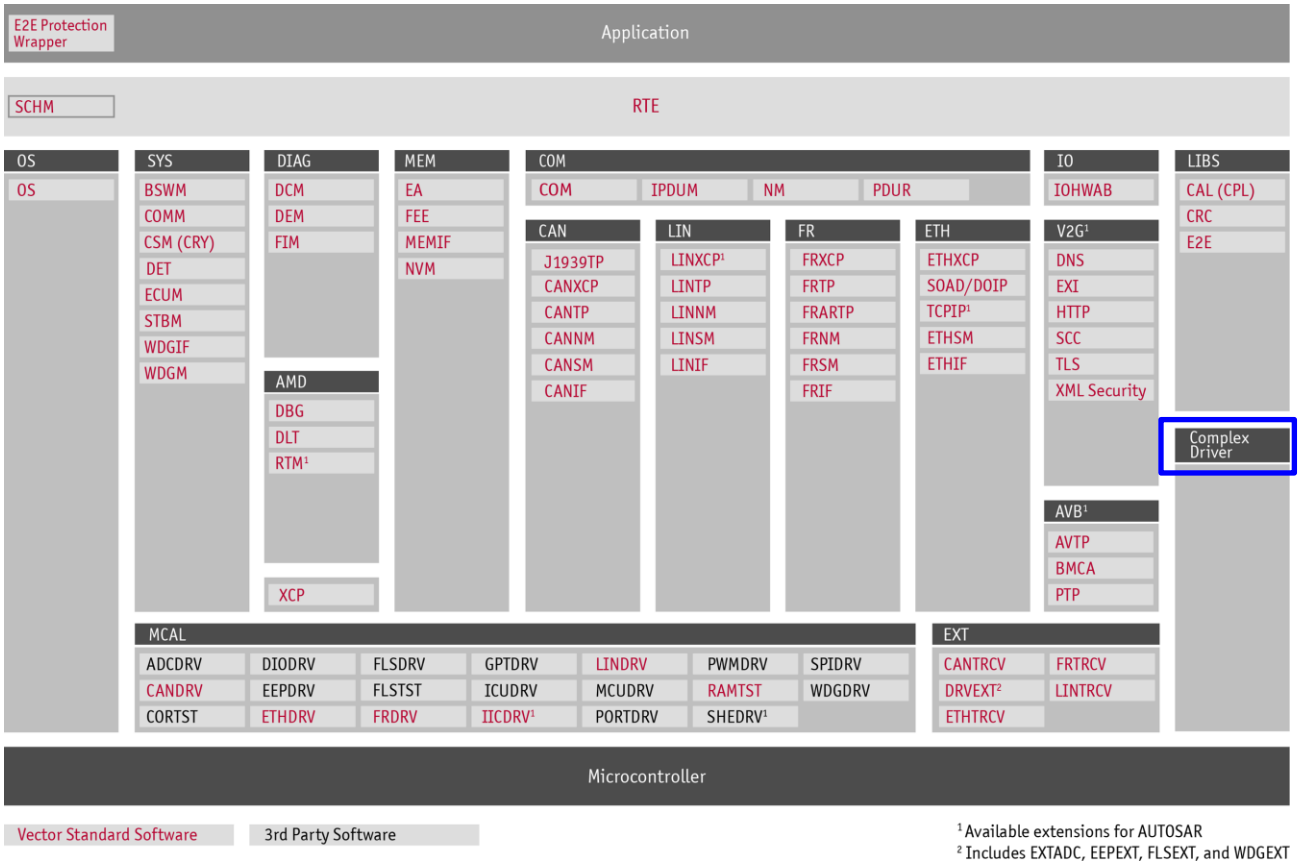


Figure 2-1 AUTOSAR 4.x Architecture Overview

The following figure shows a possible location of vMemAccM in relation to the AUTOSAR memory stack and the Software Download stack (in the figure referred to as OTA for “Over The Air”).

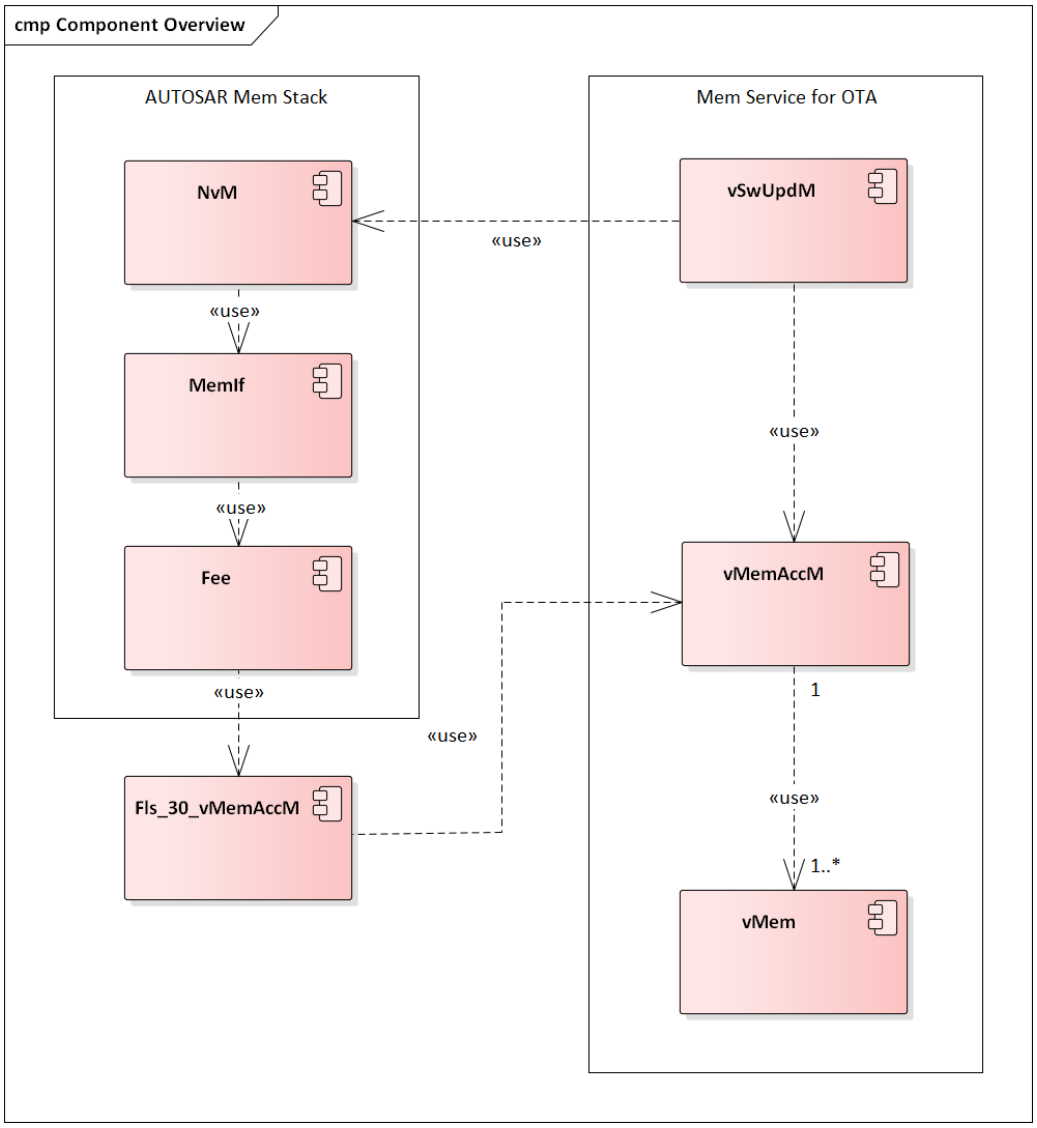


Figure 2-2 Component Overview of one most likely use case

2.2 Component Interfaces

The next figure shows the interfaces to adjacent modules of the vMemAccM. These interfaces are described in chapter 5.

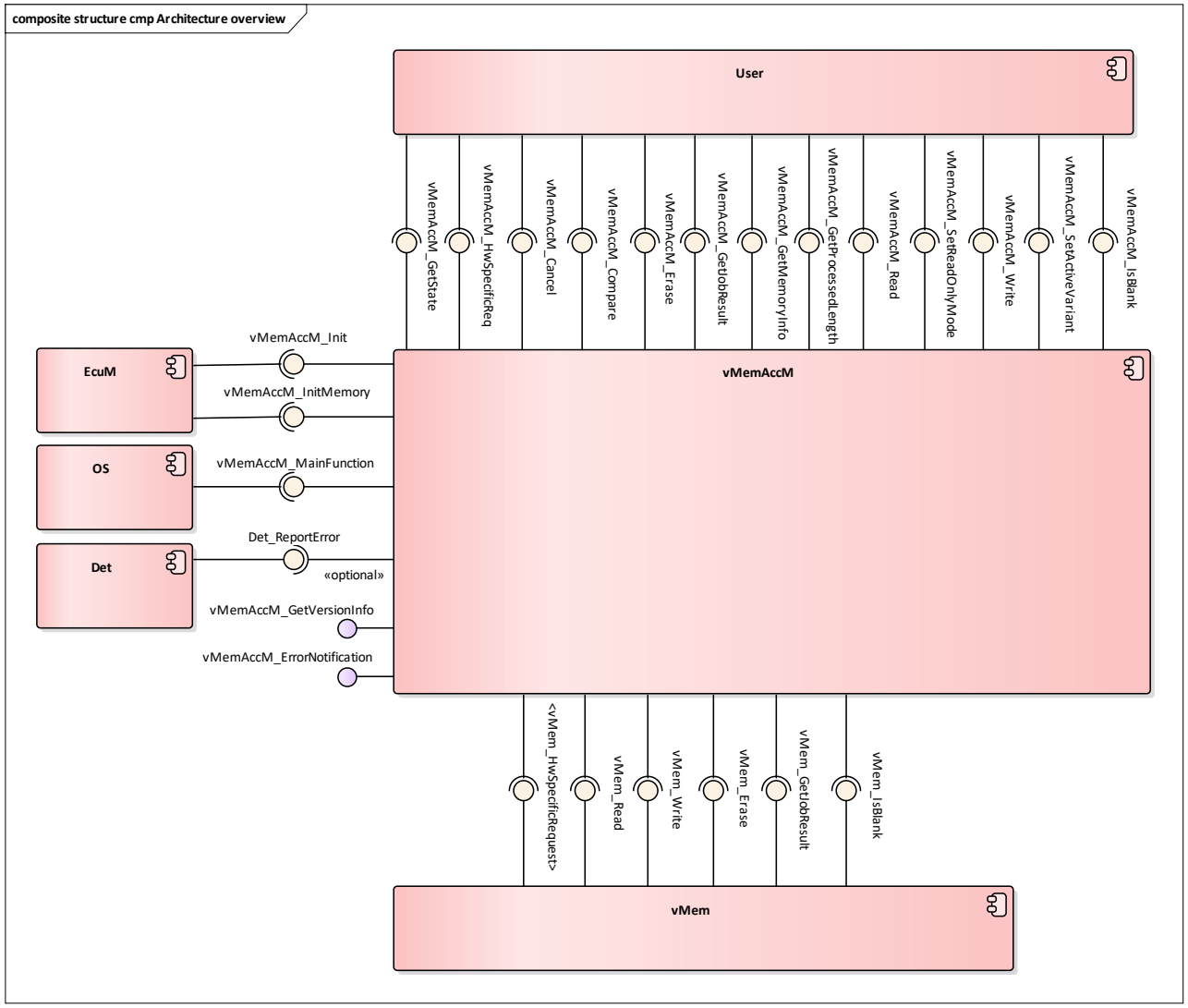


Figure 2-3 Interfaces to adjacent modules of the vMemAccM

3 Functional Description

3.1 Features

The features listed in the following table cover the complete functionality specified for the vMemAccM. Since the vMemAccM is not an AUTOSAR component all features provided are beyond AUTOSAR standard:

Provided Features
Read, erase, write, compare NV-data
Cancel an asynchronous job
Provide the result of the most recent asynchronous job
Pass through of hardware specific requests to underlying vMem modules
Set the read-only status of an address area
Set the active memory mapping variant. Support of address area memory mapping variants.
Queueing of jobs
Priority based job processing
Exploit burst capabilities of underlying driver
Provide information on memory device characteristics (e.g. the size of the smallest writable unit)
Provide information about processing state
Provide module version information
Check whether an address area is blank (erased and therefore writable)

Table 3-1 Provided Features

3.2 Initialization

Before the module vMemAccM can be used, the underlying vMem instances have to be initialized. The vMemAccM itself is initialized via the services `vMemAccM_InitMemory()` and `vMemAccM_Init()`. The internal variable structures only need to be initialized via the service `vMemAccM_InitMemory()`, in case they are not initialized by the startup code.

The EcuM usually takes care of initialization. If no EcuM is used these functions have to be called by application in correct order.

3.3 Main Functions

When an asynchronous request is accepted, the job is queued and its processing requires [1, n] `vMemAccM_MainFunction()` calls. The `vMemAccM_MainFunction()` triggering needs to be done by user (mostly OS tasks).

The following are the asynchronous API services of vMemAccM:

- `vMemAccM_Read()`
- `vMemAccM_Write()`

- `vMemAccM_Erase()`
- `vMemAccM_Compare()`
- `vMemAccM_HwSpecificReq()`
- `vMemAccM_IsBlank()`

For a more detailed description on the processing of asynchronous jobs in general, see chapters 3.7 and 3.9.



Note
The vMemAccM accepts at most one asynchronous job request per address area at a time.

3.3.1 Main Function triggering

`vMemAccM_MainFunction()` triggering can be configured to

- fixed cycle time, or
- background task.



Caution
The main function triggering must be configured to the same type within vMemAccM and all underlying vMem modules. Otherwise, for example in case of high CPU load, the background task may not be triggered, and the components might be blocking each other.

3.4 Job Results

The vMemAccM implements different job results for informing the user about the current state of an address area and the result of the latest job. This job result can be retrieved via the service `vMemAccM_GetJobResult()`.

3.4.1 VMEMACCM_JOB_PENDING

The job result is set to `VMEMACCM_JOB_PENDING` as soon as a request is accepted and then no further jobs will be accepted for the same address area. If the job result for an address area is different from `VMEMACCM_JOB_PENDING`, the address area is idle. That means, the vMemAccM does not process any job for this address area and a new job can be requested.

3.4.2 VMEMACCM_JOB_OK

The job result `VMEMACCM_JOB_OK` indicates that the last job has been processed successfully. In particular, for compare jobs it means, that the data retrieved from NV memory matches the data passed by the user.

3.4.3 VMEMACCM_READ_CORRECTED_ERRORS

The job result VMEMACCM_READ_CORRECTED_ERRORS exists for read jobs only. This result will be returned, in case ECC errors were corrected by the device at some address within the requested read range. If any of the vMem read job steps (see 3.9) returns with result VMEM_READ_CORRECTED_ERRORS, the final vMemAccM job result will never become VMEMACCM_JOB_OK. However, the requested read data is provided and valid and therefore the read job is considered successful.

3.4.4 VMEMACCM_JOB_CANCELED

The job result VMEMACCM_JOB_CANCELED indicates a previously called vMemAccM_Cancel() service for an ongoing request. The request was aborted, the respective address area is ready for the next request.

3.4.5 VMEMACCM_JOB_FAILED

The job result VMEMACCM_JOB_FAILED indicates that the last job has been processed unsuccessfully. For compare jobs it means, that the data retrieved from NV memory does not match the data passed by the user, or an actual error occurred.

3.4.6 VMEMACCM_READ_UNCORRECTABLE_ERRORS

The job result VMEMACCM_READ_UNCORRECTABLE_ERRORS exists for read jobs only. This result will be returned, in case uncorrectable ECC errors were detected by the NV memory device. Contrary to the case of corrected ECC errors, the vMemAccM will abort the read job unsuccessfully and cannot provide any valid data.

3.4.7 VMEMACCM_MEM_NOT_BLANK

The job result VMEMACCM_MEM_NOT_BLANK exists for is-blank jobs only. This result will be returned when at least a part of or the checked address space is not blank.

3.5 Error Notification

The related API (see chapter 5.2.17) is meant to be used to notify the vMemAccM about occurred ECC errors during a read job, mostly in case the vMem driver is not capable to do so. This is for example, if the related interrupt handles and therefore clears the ECC registers, so the vMem cannot read out the information about occurred ECC.

In such case the interrupt implementation shall check whether the address causing the interrupt belongs to the active/ ongoing vMemAccM job via vMemAccM_GetState() (see chapter 5.2.15) and then invoke the vMemAccM_ErrorNotification() with one of the allowed results:

- VMEMACCM_ERRORNOTIF_READ_CORRECTED_ERRORS, if the occurred ECC errors were corrected and the data can be used,
- VMEMACCM_ERRORNOTIF_READ_UNCORRECTABLE_ERRORS, if the occurred ECC leads to invalid data, vMemAccM shall abort the reading as soon as possible.

vMemAccM stores the provided result internally, waits for the vMem to finish the job and finally combines the provided and the vMem result as follows:

vMem result	Error Notification result	vMemAccM result
OK	READ_CORRECTED_ERRORS	READ_CORRECTED_ERRORS

OK	READ_UNCORRECTABLE_ERRORS	READ_UNCORRECTABLE_ERRORS
READ_CORRECTED_ERRORS	READ_CORRECTED_ERRORS	READ_CORRECTED_ERRORS
READ_CORRECTED_ERRORS	READ_UNCORRECTABLE_ERRORS	READ_UNCORRECTABLE_ERRORS
READ_UNCORRECTABLE_ERRORS	READ_CORRECTED_ERRORS	READ_UNCORRECTABLE_ERRORS
READ_UNCORRECTABLE_ERRORS	READ_UNCORRECTABLE_ERRORS	READ_UNCORRECTABLE_ERRORS
FAILED	READ_CORRECTED_ERRORS	FAILED
FAILED	READ_UNCORRECTABLE_ERRORS	FAILED

Table 3-2 vMemAccM and vMem result combinatorics in case of Error Notification.

Further vMemAccM behavior is described in chapter 3.4.



Caution
vMemAccM checks the preconditions before storing and using the passed result. These preconditions are

- Address Area identifier matches the currently active one
- Address fits to currently active address space
- The currently active job is a read job.

If one of the preconditions does not match, vMemAccM will reject the request and will not store nor use the passed result.



Basic Knowledge
Active means the vMemAccM is currently waiting for the vMem to process its job. The vMemAccM and vMem are actively working on a specific address space and therefore all e.g. ECC interrupts caused by any address belonging to the active address space are probably caused by the vMemAccM and vMem NV RAM access.

3.6 Memory Address Mapping

One of the main responsibilities of the vMemAccM is the mapping between pseudo-physical and virtual addresses, in order to provide an abstraction of the memory resources that are actually available. Upper layers only need to use virtual addresses and don’t have to be concerned with data spreading and storage on different memory devices. Whereas the underlying memory driver can only handle the actual pseudo-physical addresses.

Examples of use cases where the memory address mapping can be particularly helpful are

- one virtual address area shall be splitted into data and program flash, e.g. for an update of program and calibration data
- one virtual address area shall be splitted into different non-continuous flash sectors, e.g. for an update of different program parts

The physical memory on the level of the vMem drivers is structured in **sectors,** the smallest erasable units. Identical and coherent sectors can be grouped to **sector batches**.

This way a greater number of identical, contiguous sectors can be configured collectively and handled in the same way. In the vMemAccM the entire physical memory of all available sector batches is structured in virtual memory areas, the **address areas**. An address area contains **sub address areas** and each sub address area represents a continuous part of a physical sector batch. In other words, a sub address area is a part of an address area, which corresponds to a physical continuous area of memory.

For every sub address area a **virtual start address** is configured. The sub address areas within an address area do not have to be virtually continuous. That is, there can be gaps between the sub address areas within one address area. However, they must not overlap within the address area and if several sub address areas represent memory from the same sector batch, they must not overlap within the sector batch. The length of a sub address area is determined by the number of sectors it represents and their sector size. Note that different address areas may have the same virtual addresses.

Within every asynchronous job request an address area identifier, a virtual address within the respective address area and a length has to be passed to the vMemAccM. The vMemAccM maps the passed virtual address to a pseudo-physical address (vMem device instance and device specific address).



Caution
The address mapping can only be performed, if the whole virtual address range of the request is valid. Hence, asynchronous jobs can only be requested for virtual address space, which is filled with sub address areas. Requests that include gaps cannot be processed properly.

Figure 3-1 Memory address mapping shows an example of an address area containing four sub address areas. The virtual start address of the address area is 0x0, however its first valid address is the start address of the first sub address area. The virtual end address of the address area in this example is 0xFFFF FFFF which is also the maximum valid address. Note that the different colors of the memory areas have no meaning and are only used to underline where different sectors are mapped to. For more details on the configuration of address areas and sub address areas see chapter 6.2.

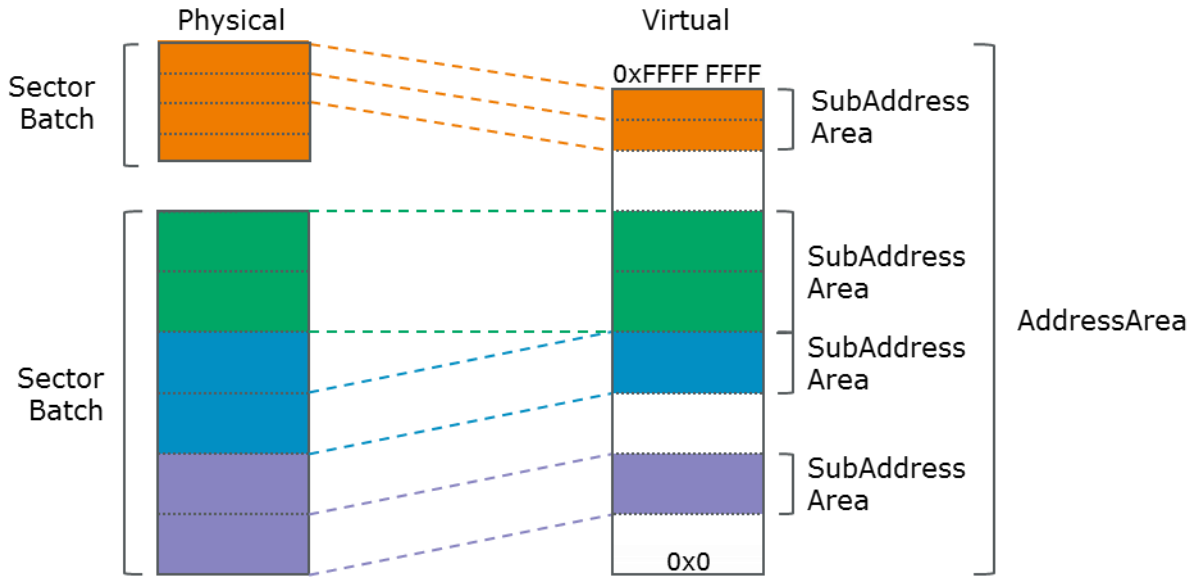


Figure 3-1 Memory address mapping

Caution
The vMemAccM checks that the whole range of an asynchronous job is valid within its internal parameter checks (see chapter 3.10).

3.6.1 Memory address mapping variants

In some use cases the virtual address needs to be remapped from one to another physical address. To achieve this the memory mapping variants were introduced.

Each address area can be configured with memory mapping variants. To set the needed memory mapping variant via appropriate API vMemAccM provides two variant identifiers:

- `VMEMACCM_VARIANT_ID_STARTUP 0u`: the startup variant, always set during the initialization (startup). Always available and valid for all Address Areas (Address Areas without variants know only one memory mapping).
- `VMEMACCM_VARIANT_ID_ALTERNATE 1u`: the alternate memory mapping. Only available, if at least one Address Area is configured with memory mapping variants.

The variant identifiers 0u and 1u are unique – 0u always refers to the startup variant, 1u to the alternate variant. The identifiers must be set within the configuration according to the needs.

Caution
The set variant is always valid for all address areas with variants. It is not possible to set the variant of each address area separately.



Basic Knowledge

- Each address area memory mapping variant spans the same virtual address space.
- Each address area spans all required sub address areas, each address area variant refers to the required sub address areas (configuration).
- An address area without variants must not configure the variant container, just the needed sub address area containers. Calling the `vMemAccM_SetActiveVariant` function will not affect these address areas.
- It is not possible to configure only one address area memory mapping variant, vMemAccM always needs two variants to switch between.

3.7 Priority Based Job Processing

The vMemAccM accepts at most one asynchronous job request per address area at a time. Since the vMemAccM will process only one asynchronous job at a time, it implements a queue of size number of address areas. An asynchronous job request is only accepted and queued if there is no other currently pending job for the requested address area. So, if for instance a read job for an address area is queued, an erase job for the same address area will not be accepted before the read job is finished.

Each address area has its own priority which can be configured. Note that the priorities of the address areas must be pairwise different. The priorities 0 (lowest priority) until 65535 (highest priority) can be selected for an address area.

An asynchronous job on an address area with higher priority is processed before asynchronous jobs on address areas with lower priority. If there is an already ongoing asynchronous job on an address area with lower priority, it will be suspended as soon as possible. Note that an already ongoing asynchronous job on the underlying vMem will not be interrupted.

3.8 Burst Feature

The underlying vMem driver may offer burst capabilities, that is instead of writing/erasing one smallest possible unit a number of these units are written/erased at once to increase performance.

The smallest writable physical memory unit is usually called a **page**. To improve performance, the underlying vMem modules may offer a **write burst** feature depending on the hardware capabilities. Instead of writing one page at a time, multiple pages within the same sector batch can be written at once.

The smallest erasable physical memory unit is usually called a **sector**. To improve performance, the underlying vMem modules may offer an **erase burst** feature, similarly as for write jobs. Instead of erasing one sector at a time, multiple sectors within the same sector batch can be erased at once.

If the underlying vMem module is burst capable, the user of the vMemAccM can chose during configuration whether the write or erase burst feature shall be exploited by the

vMemAccM. The advantage of using the burst feature is certainly the increased performance. However, the user may still not want the vMemAccM to burst, because of too long burst durations. For example, it might happen that a job with low priority blocks an address area for a job with high priority for a longer time, because burst is used.

3.9 Partitioning of Asynchronous Jobs

Note that in this document the **length of an asynchronous job** always refers to the number of bytes of data which shall be processed. This information is always passed as a parameter of an asynchronous request.

To avoid blocking requests the vMemAccM splits asynchronous job requests into smaller jobs which are forwarded to the underlying vMem modules. An asynchronous job for an address area will be divided into at least the number of involved sub address areas, since each sub address area might correspond to an entirely different sector batch. But even requests which involve only one sub address area are divided into job steps of shorter length. The step length depends on the request and configuration parameters. In the following the partitioning of the individual asynchronous jobs are described in more detail.

3.9.1 Read

The vMemAccM divides the read request into smaller jobs which are forwarded to the underlying vMem modules. Theoretically the vMem may accept read requests with any length, but this way the related vMem is only blocked for a shorter time. The maximum number of bytes the vMemAccM module requests within one vMem read request can be configured per address area (see chapter 6.2.1).

3.9.2 Write

Write jobs are partitioned into smaller jobs of length write burst size or page size.

3.9.3 Erase

Erase jobs are partitioned into smaller jobs of length erase burst size or sector size.

3.9.4 Compare

The compare job differs from the other asynchronous jobs, since the underlying vMem modules do not offer a compare service. Hence the vMemAccM cannot simply forward the compare request to the vMem but has to read the data in question using the vMem read service and store it in an internal buffer (see chapter 3.11). The vMemAccM then compares the data read to the data passed by the user. Therefore, the maximum number of bytes the vMemAccM can compare at a time is determined by the maximum read request length (of the related address area).

3.9.5 IsBlank

IsBlank jobs are partitioned into smaller jobs of page size.

3.9.6 Hardware Specific Requests

It is possible that the underlying vMem modules offers other special features to access memory besides the basic ones (read, write and erase), depending on the individual hardware. Since any kind of memory access shall pass through the vMemAccM, the vMemAccM offers a special asynchronous service `vMemAccM_HwSpecificReq()`, which allows upper layers to use these special features.

The hardware specific requests are treated by the vMemAccM as any other asynchronous job, that is no other asynchronous jobs will be accepted for the corresponding address area until the job is completed.

Information on a specific vMem hardware instance, such as the unique hardware identifier, can be obtained via the service `vMemAccM_GetMemoryInfo()`. The unique hardware identifier is needed as a parameter in the service `vMemAccM_HwSpecificReq()`.



Caution
The vMemAccM interface shall be open for any vMem implementation and cannot be responsible for partitioning the hardware specific requests into jobs of smaller length. Therefore, vMemAccM will neither check nor access the passed data pointer and its size. The values will be forwarded directly the vMem.



Note
If the vMem request is basically a void-void function, it still must match the signature. In this case the pointer can be NULL_PTR and the size of the pointer can be 0.

3.10 Retries of a failed job

To resolve transient errors vMemAccM retries a failed vMem request.

Since a vMemAccM request normally leads to several vMem requests, the retries will always be done on the one failed vMem request setting up the same request (instance identifier, physical address, buffer address and length) again and again until the vMem finishes the job successfully, or the retry count exceeds the configured maximum value.

The user will not be notified about the retrying. Retrying the job vMemAccM ensures a failed request was not caused by transient errors, but caused by a potentially real hardware problem. The user shall behave according to the available failure handlings (depends on used hardware), e.g. for a failed write the sector shall be erased and then the data written again.



Practical Procedure
Find the address causing the failure:

The failing address can be calculated via the successfully processed request length (in bytes) published via `vMemAccM_GetProcessedLength`. The length always shows the successfully processed number of bytes.

If the job result matches an unsuccessful job and the returned length is lower than the requested length, the length + 1 marks the failing address offset.



Basic Knowledge
vMemAccM does not retry rejected jobs.

vMem may reject a burst size request, where vMemAccM retries the job setup with normal size, assuming vMem cannot burst right now because e.g. not aligned RAM buffer.

But after the vMem rejects a normal size request, vMemAccM assumes the vMem is not operable and aborts processing unsuccessfully.

3.10.1 Retry a failed burst size request

A burst size vMem request consists of several pages (write) or sectors (erase). To find the most precise address of the failure, vMemAccM uses a slightly different retry strategy than for normal size requests: split the burst size into normal size jobs and process all these normal size jobs before using the burst feature again.

If there is a real hardware failure, one of the normal size requests will fail and the `vMemAccM_GetProcessedLength` function will deliver the successfully processed length as described in previous chapter.

If all the normal size requests succeed, vMemAccM will assume the failure was caused by a transient error and continue processing with the next step (normal size or burst size – depending on current job progress).

3.10.2 Retry configuration

Because the possible number of retries depends on hardware, the value must be configured within a sub address area as the mapping element from virtual to physical address space and therefore to the hardware.

The configured number of retries is always valid for one physical address, that means once a vMem request to a specific address succeeds, the next vMem request to the next address will use its own retry counter.

Number of retries 0 disables retries, vMemAccM will never retry a failed vMem request. 1 forces vMemAccM to retry a failed vMem request once, and so on.

3.10.3 Supported vMemAccM requests

- vMemAccM retries a failed write job step.

- vMemAccM retries a failed erase job step.



Caution
To use the retries, the hardware must support

> overwriting the physical content with the same data (part of the data may already be written correctly!)

> erasing an already erased physical area.

3.11 RAM Buffers

vMemAccM never copies the data to another RAM buffer before forwarding the buffer to vMem – it uses the provided buffer directly to read and write data.

Only for the compare service, the vMemAccM uses an internal buffer. vMemAccM will then forward the internal buffer to the vMem to get the data from NV RAM and compare the read data against the provided user data.

The internal buffer length matches the largest `vMemAccMReadRequestLength` (see chapter 6.2.1). If the function `vMemAccM_Compare()` is disabled via the parameter `vMemAccMCompareApi`, the internal buffer is neither reserved nor used at all.



Caution
vMemAccM does not ensure device related RAM alignments, upper layers must take care about the correctly aligned RAM start addresses.

vMemAccM uses the internal buffer only to read data from NV RAM, the buffer will never be used to write data. Therefore, the internal buffer does not fulfill any specific RAM alignments.

3.12 Multi partition synchronization

vMemAccM implements a multi partition synchronization, where multiple separate partitions (cores) access the NV RAM on their own and the vMemAccM takes care, that only one partition does so at a time.

One of the contributing partitions must be the master, all others are slaves. The master synchronizes the accesses and sets the access right to one of the partitions requesting a NV RAM access.

The current implementation does not use any operating system functionalities.

Figure 3-2 shows an example of the multi partition synchronization within the memory stack architecture.

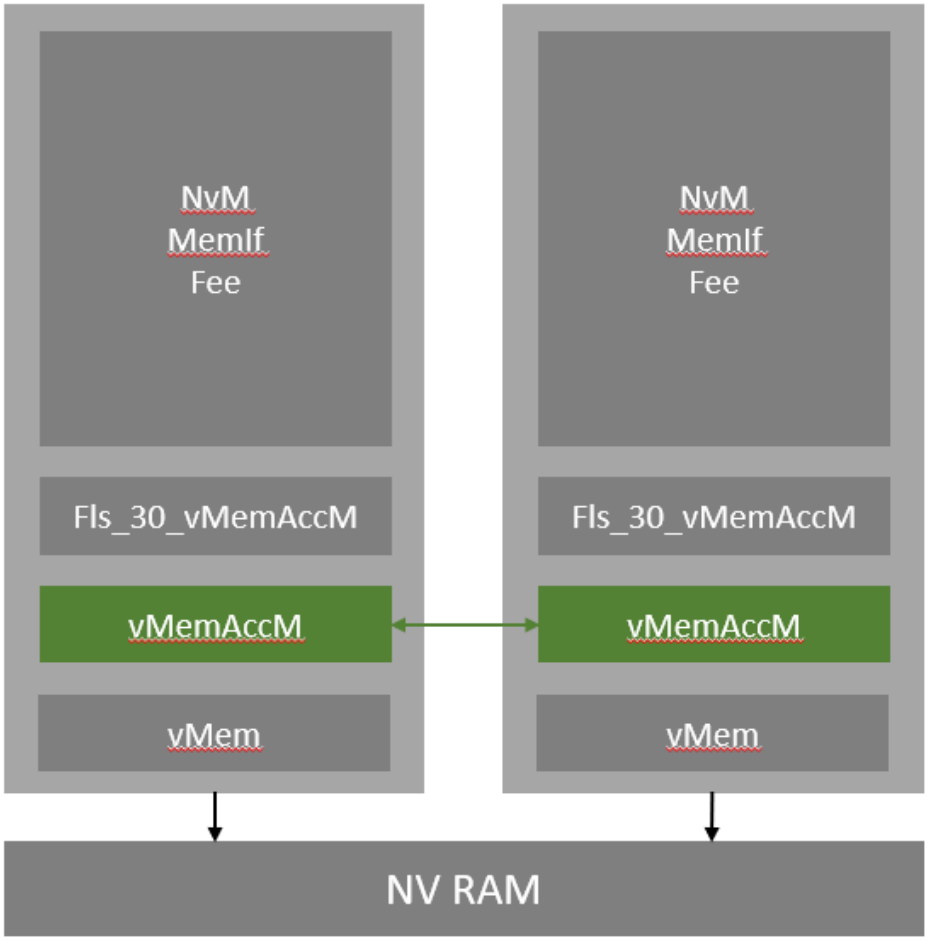


Figure 3-2 Multi partition synchronization architecture example (2 cores)

Example

If one partition/ core must keep its data locally, so the other partition cannot read the data, two memory stacks may be used in parallel.

If the Flash device does not support parallel access from two separated users, the access needs to be synchronized, which is possible via the vMemAccM.

Example of such a situation: HSM core and application core.

If both partitions can access the NV RAM in parallel, this feature is not needed!

The following chapters describe how to configure the vMemAccM module and the required variables.

3.12.1 Configuration

A vMemAccM will be the multi partition synchronization master, if the vMemAccMMasterPartitionConfiguration container exist. Otherwise the vMemAccM is a multi partition synchronization slave.

3.12.1.1 Master partition

A master, the one taking care of NV RAM access synchronization, needs to know all partitions contributing in the NV RAM access synchronization.

If the configuration contains the vMemAccMMasterPartitionConfiguration, the vMemAccM is the master and all partitions contributing in the NV RAM access synchronization need to be configured as vMemAccMPartitionConfiguration.

As master the vMemAccM automatically generates the variables needed for the synchronization:

- `vMemAccM_NVRAMAccessToken` holding the current partition identifier with the NV RAM access, or synchronization information.
- One `vMemAccM_PublishReq_<vMemAccMPartitionConfiguration><vMemAccMPartitionId>` per partition. The partition publishes its state to the master via this variable: partition wants to access the NV RAM, or is idle (no jobs requested).

These variables need to be mapped into appropriate shared RAM, see chapter 3.12.2 for more information.

Please see the next chapter for information about slave partition configuration.

3.12.1.2 Slave partitions

A slave partition, the one requesting NV RAM access and waiting for the token to hold the own partition identifier, does not declare any variables. The user needs to configure shared RAM addresses or linker symbols pointing to the variables created by the master partition.

These are:

- vMemAccMNVRAMAccessToken pointing to the `vMemAccM_NVRAMAccessToken` variable.
- vMemAccMPartitionPublishRequest pointing to the appropriate `vMemAccM_PublishReq_<vMemAccMPartitionConfiguration><vMemAccMPartitionId>` variable.

A slave vMemAccM partition does not contain the vMemAccMMasterPartitionConfiguration container. Since the master configuration always requires all partitions, a slave configuration shall always match one of the partitions available in master configuration!

The vMemAccMNVRAMAccessToken, vMemAccMPartitionPublishRequest and also the vMemAccMPartitionId must match the master configuration.

3.12.2 Memory Mapping

The multi partition synchronization variables declared by the vMemAccM master partition must be mapped into appropriate shared RAM region(s). The vMemAccM memory mapping sections always follow the naming schema:

- VMEMACCM_START_SEC_MULTIPARTITION_PARTITION_ID<PartitionId>
- VMEMACCM_STOP_SEC_MULTIPARTITION_PARTITION_ID<PartitionId>.

The sections always hold the following variables:

- VMEMACCM_START/STOP_SEC_MULTIPARTITION_PARTITION_ID0
  - The NV RAM access token the master partition writes, all other partition read.
- VMEMACCM_START/STOP_SEC_MULTIPARTITION_PARTITION_ID<1…n>
  - The request value published/ written by the respective partition, the master partition reads the value.

If the memory mapping sections are unmapped, for example after configuring the vMemAccM synchronization for the very first time, the vMemAccM memory sections will cause a compiler error. This will be done to notify the integrator about the missing memory mapping pragmas.

3.12.2.1 Examples

Following sections describe how the mapping can be done for specific compilers. These are just examples, please refer to the appropriate compiler documentation for more information.

Example: Windriver DiabData Compiler

```
#ifdef VMEMACCM_START_SEC_MULTIPARTITION_PARTITION_ID0
# undef VMEMACCM_START_SEC_MULTIPARTITION_PARTITION_ID0
# pragma section DATA ".vMemAccMNVRAMAccessToken" ".vMemAccMNVRAMAccessToken"
# define START_SEC_VAR_INIT_32BIT
#endif

#ifdef VMEMACCM_STOP_SEC_MULTIPARTITION_PARTITION_ID0
# undef VMEMACCM_STOP_SEC_MULTIPARTITION_PARTITION_ID0
# pragma section DATA
# define STOP_SEC_VAR
#endif

#ifdef VMEMACCM_START_SEC_MULTIPARTITION_PARTITION_ID<PartitionId>
# undef VMEMACCM_START_SEC_MULTIPARTITION_PARTITION_ID<PartitionId>
# pragma section DATA ".vMemAccMPublishRequest<PartitionId>"
".vMemAccMPublishRequest<PartitionId>"
# define START_SEC_VAR_INIT_32BIT
#endif

#ifdef VMEMACCM_STOP_SEC_MULTIPARTITION_PARTITION_ID<PartitionId>
# undef VMEMACCM_STOP_SEC_MULTIPARTITION_PARTITION_ID<PartitionId>
# pragma section DATA
# define STOP_SEC_VAR
#endif
```

Example: Green Hills Compiler

```
#ifdef VMEMACCM_START_SEC_MULTIPARTITION_PARTITION_ID0
# undef VMEMACCM_START_SEC_MULTIPARTITION_PARTITION_ID0
# pragma ghs section data=".vMemAccMNVRAMAccessToken"
# pragma ghs section sdata=".svMemAccMNVRAMAccessToken""
# define START_SEC_VAR_INIT_32BIT
#endif

#ifdef VMEMACCM_STOP_SEC_MULTIPARTITION_PARTITION_ID0
# undef VMEMACCM_STOP_SEC_MULTIPARTITION_PARTITION_ID0
# pragma ghs section data=default
# pragma ghs section sdata=default
# define STOP_SEC_VAR
#endif

#ifdef VMEMACCM_START_SEC_MULTIPARTITION_PARTITION_ID<PartitionId>
# undef VMEMACCM_START_SEC_MULTIPARTITION_PARTITION_ID<PartitionId>
# pragma ghs section data=".vMemAccMPublishRequest<PartitionId>"
# pragma ghs section sdata=".svMemAccMPublishRequest<PartitionId>"
# define START_SEC_VAR_INIT_32BIT
#endif

#ifdef VMEMACCM_STOP_SEC_MULTIPARTITION_PARTITION_ID<PartitionId>
# undef VMEMACCM_STOP_SEC_MULTIPARTITION_PARTITION_ID<PartitionId>
# pragma ghs section data=default
# pragma ghs section sdata=default
# define STOP_SEC_VAR
```

```
#endif
```



Example: Altium TASKING Compiler

```
#ifdef VMEMACCM_START_SEC_MULTIPARTITION_PARTITION_ID0
# undef VMEMACCM_START_SEC_MULTIPARTITION_PARTITION_ID0
# pragma section all “vMemAccMNVRAMAccessToken”
# define START_SEC_VAR_INIT_32BIT
#endif

#ifdef VMEMACCM_STOP_SEC_MULTIPARTITION_PARTITION_ID0
# undef VMEMACCM_STOP_SEC_MULTIPARTITION_PARTITION_ID0
# pragma section all restore
# define STOP_SEC_VAR
#endif

#ifdef VMEMACCM_START_SEC_MULTIPARTITION_PARTITION_ID<PartitionId>
# undef VMEMACCM_START_SEC_MULTIPARTITION_PARTITION_ID<PartitionId>
# pragma section all “.vMemAccMPublishRequest<PartitionId>”
# define START_SEC_VAR_INIT_32BIT
#endif

#ifdef VMEMACCM_STOP_SEC_MULTIPARTITION_PARTITION_ID<PartitionId>
# undef VMEMACCM_STOP_SEC_MULTIPARTITION_PARTITION_ID<PartitionId>
# pragma section all restore
# define STOP_SEC_VAR
#endif
```

3.13 Error Handling

3.13.1 Development Error Reporting

By default, development errors are reported to the DET using the service `Det_ReportError()` as specified in [1], if development error reporting is enabled (i.e. pre-compile parameter `VMEMACCM_DEV_ERROR_DETECT==STD_ON`).

If another module is used for development error reporting, the function prototype for reporting the error can be configured by the integrator but must have the same signature as the service `Det_ReportError()`.

The reported vMemAccM ID is 255.

The reported service IDs identify the services which are described in detail in 0. The following table presents the service IDs and the related services:

Service ID	Service
0x00	`vMemAccM_InitMemory()`
0x01	`vMemAccM_Init()`
0x02	`vMemAccM_Read()`
0x03	`vMemAccM_Write()`
0x04	`vMemAccM_Erase()`

Service ID	Service
0x05	vMemAccM_Compare()
0x06	vMemAccM_HwSpecificReq()
0x07	vMemAccM_MainFunction()
0x08	vMemAccM_Cancel()
0x09	vMemAccM_GetJobResult()
0x0A	vMemAccM_GetMemoryInfo()
0x0B	vMemAccM_SetReadOnlyMode()
0x0C	vMemAccM_GetProcessedLength()
0x0D	vMemAccM_GetVersionInfo()
0x0E	vMemAccM_GetState()
0x0F	vMemAccM_SetActiveVariant()
0x10	vMemAccM_ErrorNotification()
0x11	vMemAccM_IsBlank()

Table 3-3 Service IDs

The errors reported to DET are described in the following table:

Error Code		Description
0x01	VMEMACCM_E_UNINIT	An API service has been called before component was initialized.
0x02	VMEMACCM_E_PARAM_POINTER	An API service has been called with an NULL pointer.
0x03	VMEMACCM_E_PARAM_ADDRESS_AREA_ID	An API service has been called with an invalid address area ID.
0x04	VMEMACCM_E_PARAM_ADDRESS_LENGTH	An API service has been called with an invalid virtual address and / or length. For instance, the virtual address is not within the related address area or the requested length exceeds the length of the address area.
0x05	VMEMACCM_E_PENDING	An asynchronous API service was called, while another job of requested address area is already being processed. Only one asynchronous job request is accepted per address area.
0x06	VMEMACCM_E_PARAM_HW_SPECIFIC_FUNC_INDEX	The passed HW specific function index does not belong to passed address area ID.
0x07	VMEMACCM_E_PARAM_HW_ID	The passed HW identifier does not belong to passed address area ID.
0x08	VMEMACCM_E_PARAM_VARIANT_ID	The passed memory mapping variant identifier does not exist.
0x09	VMEMACCM_E_ADDRESS_AREA_NOT_ACTIVE	The passed address area ID is not active in processing sub module.

Table 3-4 Errors reported to DET

3.13.1.1 Parameter Checking

The following table shows which parameter checks are performed on which services:

Service \ Check/Error code	VMEMACCM_E_UNINIT	VMEMACCM_E_PARAM_POINTER	VMEMACCM_E_PARAM_ADDRESS_AREA_ID	VMEMACCM_E_PARAM_ADDRESS_LENGTH	VMEMACCM_E_PENDING	VMEMACCM_E_PARAM_HW_SPECIFIC_FUNC_INDEX	VMEMACCM_E_PARAM_HW_ID	VMEMACCM_E_PARAM_VARIANT_ID	VMEMACCM_E_ADDRESS_AREA_NOT_ACTIVE
vMemAccM_InitMemory()									
vMemAccM_Init()									
vMemAccM_Read()	■	■	■	■	■				
vMemAccM_Write()	■	■	■	■	■				
vMemAccM_Erase()	■		■	■	■				
vMemAccM_Compare()	■	■	■	■	■				
vMemAccM_HwSpecificReq()	■	■	■		■	■	■		
vMemAccM_MainFunction()	■								
vMemAccM_Cancel()	■		■						
vMemAccM_GetJobResult()	■		■						
vMemAccM_GetMemoryInfo()	■	■	■						
vMemAccM_SetReadOnlyMode()	■		■						
vMemAccM_GetProcessedLength()	■	■	■						
vMemAccM_GetVersionInfo()		■							
vMemAccM_IsBlank()	■		■	■	■				
vMemAccM_SetActiveVariant()	■				■			■	
vMemAccM_ErrorNotification()	■		■						■

Table 3-5 Development Error Reporting: Assignment of checks to services

4 Integration

This chapter gives necessary information for the integration of the MICROSAR vMemAccM into an application environment of an ECU.

4.1 Scope of Delivery

The delivery of the vMemAccM contains the files which are described in the chapters 4.1.1 and 4.1.2:

4.1.1 Static Files

File Name	Description
vMemAccM.c	This is the source file of the vMemAccM. It contains the implementation of the vMemAccM interfaces.
vMemAccM.h	This is the header file of the vMemAccM. It declares the interfaces of vMemAccM.
vMemAccM_Types.h	This file defines general types used by vMemAccM.
vMemAccM_vMemApi.h	This file defines the interfaces of all functions which must be provided by the vMem modules so that vMemAccM can use them. All vMem instances must implement these functions.
vMemAccM_MemAb.h vMemAccM_MemAb.c vMemAccM_Proc.h vMemAccM_Proc.c vMemAccM_Queue.h vMemAccM_Queue.c	These are files for internal use of the vMemAccM.

Table 4-1 Static files

4.1.2 Dynamic Files

The dynamic files are generated by the configuration tool DaVinci Configurator 5.

File Name	Description
vMemAccM_Cfg.h	This file contains the public configuration parameters of the vMemAccM.
vMemAccM_Lcfg.c	This file contains configuration parameters.
vMemAccM_vMemCfg.h	This file contains the `vMemAccM_HwIdType`.

Table 4-2 Generated files

4.1.3 Interfaces of the vMem

As described in chapter 4.1.1 the vMemAccM contains a file vMemAccM_vMemApi.h specifying the interfaces of the vMem. In addition the vMemAccM contains the description file vMem_Base_bswmd.arxml which must be refined by every vMem module.

4.2 Critical Sections

To protect critical code against interruptions vMemAccM uses the following exclusive area:

> `VMEMACCM_EXCLUSIVE_AREA_0`

The exclusive area covers only some statements and shall have a short runtime. The critical section is used vMemAccM internally and does not cover calls to other modules. vMemAccM does not nest critical sections.

© 2020 Vector Informatik GmbH

5 API Description

For an interfaces overview please see Figure 2-3.

5.1 Type Definitions

The types defined by the vMemAccM are described in this chapter.

Type Name	C-Type	Description	Value Range
vMemAccM_AddressAreaIdType	uint16	Type for the unique numeric ID of an AddressArea. The IDs are generated as defines using the symbolic name value.	0 – 65535
vMemAccM_AddressType	uint32	Type for the virtual address within an AddressArea.	0 – 4294967295
vMemAccM_LengthType	uint32	Type for the length of an asynchronous job.	0 – 4294967295
vMemAccM_vMemHwSpecificFuncIndexType	uint32	Type for index of HW specific function within HW specific function table.	0 – 4294967295
vMemAccM_LengthPtrType	uint32*	Type used to return the processed length of an asynchronous job.	Must not be NULL pointer.
vMemAccM_DataPtrType	void*	Type for pointer to buffer passed by user in read request where vMemAccM shall write the requested memory content to.	Must not be NULL pointer.
vMemAccM_ConstDataPtrType	void*	Type for pointer to buffer passed by user in write request.	Must not be NULL pointer.
vMemAccM_JobResultType	enum	Type for the asynchronous job results. The result can be polled via vMemAccM_GetJobResult() for every AddressArea and can have the following values.	VMEMACC_JOB_OK The last asynchronous job has been finished successfully. This is the default value after initialization.
			VMEMACC_JOB_PENDING An asynchronous job is currently being processed or within queue.
			VMEMACC_JOB_CANCELED The last asynchronous job has been canceled by vMemAccM_Cancel().

Type Name	C-Type	Description	Value Range
			VMEMACC_JOB_FAILED The last asynchronous job has been finished unsuccessfully.
			VMEMACC_READ_CORRECTED_ERRORS ECC errors were detected and corrected during the last read job.
			VMEMACC_READ_UNCORRECTABLE_ERRORS Uncorrectable ECC errors were detected during the last read job.
			VMEMACCM_MEM_NOT_BLANK The passed address area is not blank.
vMemAccM_ErrorNotifResultType	enum	Type providing all possible values for vMemAccM_ErrorNotification().	VMEMACCM_ERRORNOTIF_READ_CORRECTED_ERRORS ECC errors were detected and corrected during the last read job.
			VMEMACCM_ERRORNOTIF_READ_UNCORRECTABLE_ERRORS Uncorrectable ECC errors were detected during the last read job.
vMemAccM_HwIdType	enum	Type for the unique numeric identifiers of all vMem hardware instances used for hardware specific requests.	0 – 4294967295 The name of each enum parameter is constructed from the vMem module name and the vMem instance name.
vMemAccM_JobType	enum	Type for the asynchronous jobs – lists all available asynchronous jobs.	VMEMACCM_NO_JOB
			VMEMACCM_WRITEJOB
			VMEMACCM_READJOB
			VMEMACCM_COMPAREJOB
			VMEMACCM_ERASEJOB
			VMEMACCM_VMEMJOB
vMemAccM_VariantIdType	uint8	Type for unique numeric address area memory mapping variant identifier.	0-255

Table 5-1 Type definitions

vMemAccM_MemoryInfoType

This structure contains information on memory device characteristics and can be accessed via the vMemAccM_GetMemoryInfo() service. The elements of vMemAccM_MemoryInfoType are described in the table below.

Struct Element Name	C-Type	Description	Value Range
VirtualStartAddress	uint32	Virtual start address of sub address area	0 – 4294967295
PhysicalStartAddress	uint32	Physical start address of sub address area	0 – 4294967295
MaxOffset	uint32	The maximum offset from both start addresses - virtual and physical. (== sub address area length - 1)	0 – 4294967295
NumberOfSectors	uint32	Number of sectors of sub address area	0 – 4294967295
SectorSize	uint32	Size of a sector in bytes	0 – 4294967295
SectorBurstSize	uint32	Size of a sector burst in bytes. Equals SectorSize in case burst is disabled.	0 – 4294967295
PageSize	uint32	Size of a page in bytes	0 – 4294967295
PageBurstSize	uint32	Size of a page burst in bytes. Equals PageSize in case burst is disabled.	0 – 4294967295
PageBurstRamAlignment	uint32	Alignment for page burst RAM, 1 if burst disabled.	0 – 4294967295
HwId	enum: vMemAccM_HwIdType	Referenced vMem hardware identifier	0 – 4294967295

Table 5-2 vMemAccM_MemoryInfoType

vMemAccM_StateType

This structure contains information the current processing state of the vMemAccM.

Struct Element Name	C-Type	Description	Value Range
Address	uint32	Address of the currently active address area request.	0 – 4294967295
Length	uint32	Length of the currently active address area request.	0 – 4294967295
vMemInstanceId	uint32	Instance ID of the current vMem request.	0 – 4294967295
vMemAddress	uint32	Physical address of the current vMem request.	0 – 4294967295
vMemLength	uint32	Length of the current vMem request.	0 – 4294967295
CurrentJob	vMemAccM_JobType	The job vMemAccM processes for the active address area.	-
vMemResult	vMemAccM_JobResultType	Current or last vMem result.	-
AddressAreaId	uint32	Id of the active address area.	0 – 65535

Table 5-3 vMemAccM_StateType

5.2 Services provided by vMemAccM

5.2.1 vMemAccM_InitMemory

Prototype	
`void vMemAccM_InitMemory (void)`	
Parameter	
void	none
Return code	
void	none
Functional Description	
Function for *_INIT_*-variable initialization. Service to initialize module global variables at power up. This function initializes the variables in *_INIT_* sections. Used in case they are not initialized by the startup code.	
Particularities and Limitations	
-	
Call context	
> TASK (usually start-up sequence) > This function is synchronous. > This function is non-reentrant.	

Table 5-4 vMemAccM_InitMemory

5.2.2 vMemAccM_Init

Prototype	
`void vMemAccM_Init (void)`	
Parameter	
void	none
Return code	
void	none
Functional Description	
Initialization function. This function initializes the module vMemAccM. It initializes all variables and sets the module state to initialized.	
Particularities and Limitations	
> `vMemAccM_InitMemory` has been called unless `vMemAccM_ModuleInitialized` is initialized by start-up code.	
Call context	
> TASK (usually ECU initialization sequence e.g. EcuM driver Init-phases) > This function is synchronous. > This function is non-reentrant.	

Table 5-5 vMemAccM_Init

5.2.3 vMemAccM_Read

Prototype	
`Std_ReturnType` **`vMemAccM_Read`** `(vMemAccM_AddressAreaIdType AddressAreaId, vMemAccM_AddressType SourceAddress, vMemAccM_DataPtrType DestinationPtr, vMemAccM_LengthType Length)`	
Parameter	
AddressAreaId [in]	Numeric identifier of address area.
SourceAddress [in]	Virtual address within address area from which the read job shall start.
DestinationPtr [out]	Pointer to where the read data shall be stored.
Length [in]	Number of bytes that shall be read.
Return code	
Std_ReturnType	E_OK job accepted and queued, E_NOT_OK otherwise
Functional Description	
Reads NV memory content of the related address area to passed buffer – from address with length number of bytes. Queues the job, which will be processed during `vMemAccM_MainFunction`. Its status can be polled via `vMemAccM_GetJobResult`.	
Particularities and Limitations	
-	
Call context	
> TASK > This function is asynchronous. > This function is reentrant.	

Table 5-6 vMemAccM_Read

5.2.4 vMemAccM_Write

Prototype	
`Std_ReturnType` **`vMemAccM_Write`** `(vMemAccM_AddressAreaIdType AddressAreaId, vMemAccM_AddressType TargetAddress, vMemAccM_ConstDataPtrType SourcePtr, vMemAccM_LengthType Length)`	
Parameter	
AddressAreaId [in]	Numeric identifier of address area.
TargetAddress [in]	Virtual address within address area from which write job shall start.
SourcePtr [in]	Pointer to the data which shall be written to memory.
Length [in]	Number of bytes that shall be written. Keep in mind, that the memory is only page-wise writeable. Hence requested length must always be aligned to page size.
Return code	
Std_ReturnType	E_OK job accepted and queued, E_NOT_OK otherwise

Functional Description
Writes the passed data to NV memory of the related address area – from address with length number of bytes. Queues the job, which will be processed during `vMemAccM_MainFunction`. Its status can be polled via `vMemAccM_GetJobResult`. If enabled for write jobs, vMemAccM uses burst capabilities of underlying vMem.
Particularities and Limitations
-
Call context
> TASK > This function is asynchronous. > This function is reentrant.

Table 5-7 vMemAccM_Write

5.2.5 vMemAccM_Erase

Prototype
`Std_ReturnType` **`vMemAccM_Erase`** `(vMemAccM_AddressAreaIdType AddressAreaId, vMemAccM_AddressType TargetAddress, vMemAccM_LengthType Length)`
Parameter

AddressAreaId [in]	Numeric identifier of address area.
TargetAddress [in]	Virtual address within address area from which erase job shall start.
Length [in]	Number of bytes that shall be erased. Keep in mind, that the memory is only sector-wise erasable. Hence requested length must always be aligned to sector size.
Return code	
Std_ReturnType	E_OK job accepted and queued, E_NOT_OK otherwise

Functional Description
Erases NV memory content of the related address area – from requested address with length number of bytes. Queues the job, which will be processed during `vMemAccM_MainFunction`. Its status can be polled via `vMemAccM_GetJobResult`. If enabled for write jobs, vMemAccM uses burst capabilities of underlying vMem.
Particularities and Limitations
-
Call context
> TASK > This function is asynchronous. > This function is reentrant.

Table 5-8 vMemAccM_Erase

5.2.6 vMemAccM_Compare

Prototype	
`Std_ReturnType` **`vMemAccM_Compare`** `(vMemAccM_AddressAreaIdType AddressAreaId, vMemAccM_AddressType SourceAddress, vMemAccM_ConstDataPtrType DataPtr, vMemAccM_LengthType Length)`	
Parameter	
AddressAreaId [in]	Numeric identifier of address area.
SourceAddress [in]	Virtual address within address area from which compare job shall start.
DataPtr [in]	Pointer to user data which shall be compared to data in NV memory.
Length [in]	Number of bytes that shall be compared.
Return code	
Std_ReturnType	E_OK job accepted and queued, E_NOT_OK otherwise
Functional Description	
Compares the passed data to NV memory content of related address area – from address with length number of bytes. Queues the job, which will be processed during `vMemAccM_MainFunction`. Its status can be polled via `vMemAccM_GetJobResult`.	
Particularities and Limitations	
`VMEMACCM_COMPARE_API == STD_ON`	
Call context	
> TASK > This function is asynchronous. > This function is reentrant.	

Table 5-9 vMemAccM_Compare

5.2.7 vMemAccM_IsBlank

Prototype	
`Std_ReturnType` **`vMemAccM_IsBlank`** `(vMemAccM_AddressAreaIdType AddressAreaId, vMemAccM_AddressType TargetAddress, vMemAccM_LengthType Length)`	
Parameter	
AddressAreaId [in]	Numeric identifier of address area. Used to access the virtual information.
TargetAddress [in]	Virtual address within address area pointing to the first byte of the request.
Length [in]	Number of bytes that shall be checked. Value is hardware dependent and must be aligned to the smallest checkable size, usually a page size.
Return code	
Std_ReturnType	E_OK job accepted and queued, E_NOT_OK otherwise
Functional Description	
Checks the passed address space to be blank, i.e. erased and writable.	

Queues the job, which will be processed during `vMemAccM_MainFunction`. Its status can be polled via `vMemAccM_GetJobResult`.
Particularities and Limitations
–
Call context
> TASK > This function is asynchronous. > This function is reentrant.

Table 5-10 vMemAccM_IsBlank

5.2.8 vMemAccM_HwSpecificReq

Prototype

```
Std_ReturnType vMemAccM_HwSpecificReq (
vMemAccM_AddressAreaIdType AddressAreaId,
vMemAccM_HwIdType HwId,
vMemAccM_vMemHwSpecificFuncIndexType vMemHwSpecificFuncIndex,
vMemAccM_DataPtrType DataPtr,
vMemAccM_LengthType Length)
```

Parameter	
AddressAreaId [in]	AddressAreaId as array index, used to access the virtual information.
HwId [in]	HwId pointing to the hardware driver the job shall be forwarded to.
vMemHwSpecificFuncIndex [in]	MemHwSpecificFuncIndex as an array index pointing to the hardware specific function pointer.
DataPtr [in,out]	DataPtr pointing to the job buffer. Value can be NULL_PTR, if not needed (DataSize == 0). If != NULL_PTR, it must stay valid until the AddresAreaId is idle (interface between vMem and upper layer, vMemAccM does not use this pointer).
Length [in]	DataSize of the data in bytes. Value can be 0, if DataPtr is NULL_PTR.
Return code	
Std_ReturnType	E_OK job accepted and queued, E_NOT_OK otherwise.

Functional Description

Sets up a hardware specific request.

vMemAccM handles the requests in the same way as any other asynchronous request, except it does not know anything about the invoked vMem function – vMemAccM just invokes one vMem specific function indexed by the passed vMemFuncIndex from provided function pointer table.

Queues the job, which will be processed during `vMemAccM_MainFunction`. Its status can be polled via `vMemAccM_GetJobResult`.

Particularities and Limitations

See vMemAccM_vMemApi.h for more information on how to implement and use vMem specific requests.

A precondition is that vMem provides the requested functionality.

Call context

> TASK

> This function is asynchronous. > This function is reentrant.

Table 5-11 vMemAccM_HwSpecificReq

5.2.9 vMemAccM_MainFunction

Prototype	
`void vMemAccM_MainFunction (void)`	
Parameter	
void	none
Return code	
void	none
Functional Description	
Takes care about job processing.	
Particularities and Limitations	
-	
Call context	
> TASK > This function is synchronous. > This function is non-reentrant.	

Table 5-12 vMemAccM_MainFunction

5.2.10 vMemAccM_Cancel

Prototype	
`void vMemAccM_Cancel (vMemAccM_AddressAreaIdType AddressAreaId)`	
Parameter	
AddressAreaId [in]	Numeric identifier of address area.
Return code	
void	none
Functional Description	
Cancels the pending job for the address area related to passed address area ID. Canceling effects only address areas in pending state, otherwise the request will be ignored.	
Particularities and Limitations	
-	
Call context	
> TASK > This function is synchronous. > This function is non-reentrant.	

Table 5-13 vMemAccM_Cancel

5.2.11 vMemAccM_GetJobResult

Prototype	
`vMemAccM_JobResultType` **`vMemAccM_GetJobResult`** `(vMemAccM_AddressAreaIdType AddressAreaId)`	
Parameter	
AddressAreaId [in]	Numeric identifier of address area.
Return code	
vMemAccM_JobResultType	Most recent job result of the passed address area ID.
Functional Description	
Returns the job result of the address area related to passed address area ID.	
Particularities and Limitations	
-	
Call context	
> TASK > This function is synchronous. > This function is reentrant.	

Table 5-14 vMemAccM_GetJobResult

5.2.12 vMemAccM_GetMemoryInfo

Prototype	

`Std_ReturnType` **`vMemAccM_GetMemoryInfo`** `(`
`vMemAccM_AddressAreaIdType AddressAreaId,`
`vMemAccM_AddressType Address,`
`vMemAccM_LengthType Length,`
`vMemAccM_MemoryInfoPtrType MemoryInfoPtr)`	

Parameter	
AddressAreaId [in]	Numeric identifier of address area.
Address [in]	Virtual address within address area from which corresponding memory device information shall be retrieved.
Length [in]	Number of bytes which shall be covered.
MemoryInfoPtr [out]	Pointer to where the memory device information shall be stored.
Return code	
Std_ReturnType	E_OK job accepted, E_NOT_OK job not accepted, or the passed address space is not continuous.
Functional Description	
Writes memory device information related to passed address area ID and address range to the passed pointer. GetMemoryInfo checks the passed address and length to cover a continuous memory, otherwise the request will be rejected. A detailed description of the retrieved information can be found in Table 5-2 vMemAccM_MemoryInfoType If the address range/ space covers more than one sub address area, vMemAccM will write the memory information of the first sub address area. The user has to increase his address to get the memory information of the next sub address area and so on.	

Particularities and Limitations
-
Call context
> TASK > This function is synchronous. > This function is reentrant.

Table 5-15 vMemAccM_GetMemoryInfo

5.2.13 vMemAccM_SetReadOnlyMode

Prototype	
`Std_ReturnType` **`vMemAccM_SetReadOnlyMode`** `(vMemAccM_AddressAreaIdType AddressAreaId, boolean ReadOnly)`	
Parameter	
AddressAreaId [in]	Numeric identifier of address area.
ReadOnly [in]	`TRUE` -> address area is set to read only `FALSE` -> address area is set to writable
Return code	
Std_ReturnType	E_OK request accepted, address area is read only, E_NOT_OK otherwise
Functional Description	
Sets the address area related to passed address area ID to read only or writable. ReadOnly == `TRUE`: vMemAccM will only accept read jobs for the related address area, other jobs will be rejected. ReadOnly == `FALSE`: vMemAccM will accept all jobs. This is the default value after initialization.	
Particularities and Limitations	
-	
Call context	
> TASK > This function is synchronous. > This function is reentrant.	

Table 5-16 vMemAccM_SetReadOnlyMode

5.2.14 vMemAccM_GetProcessedLength

Prototype	
`Std_ReturnType` **`vMemAccM_GetProcessedLength`** `(vMemAccM_AddressAreaIdType AddressAreaId, vMemAccM_LengthPtrType LengthPtr)`	
Parameter	
AddressAreaId [in]	Numeric identifier of address area.
LengthPtr [out]	Accumulated number of bytes that have already been processed of most recent job.

Return code	
Std_ReturnType	E_OK request accepted, E_NOT_OK otherwise
Functional Description	
Returns the number of bytes processed during the most recent job for passed address area ID (for more information see chapter 3.8).	
Particularities and Limitations	
-	
Call context	
> TASK > This function is synchronous. > This function is reentrant.	

Table 5-17 vMemAccM_GetProcessedLength

5.2.15 vMemAccM_GetState

Prototype	
`Std_ReturnType vMemAccM_GetState (vMemAccM_StatePtrType StatePtr)`	
Parameter	
StatePtr [out]	Pointer to a structure where vMemAccM writes the state information in.
Return code	
Std_ReturnType	E_OK provided structure was filled with state information; E_NOT_OK otherwise
Functional Description	
Returns the current processing state of the vMemAccM. The user can differ between • vMemAccM idle (not processing any job) • vMemAccM busy internal (preparing or finalizing a job, vMem is idle) • vMemAccM busy, vMem busy (vMemAccM waits for the vMem to process its job). vMemAccM provides the job type, address area, virtual address space, vMem result and the physical address space information. This service shall be used, if deeper vMemAccM processing information is necessary than provided by the GetJobResult API. For example in case the user has to know if and which NV RAM device is currently in use.	
Particularities and Limitations	
-	
Call context	
> TASK > This function is synchronous. > This function is reentrant.	

Table 5-18 vMemAccM_GetState

5.2.16 vMemAccM_SetActiveVariant

Prototype	
`Std_ReturnType` **`vMemAccM_SetActiveVariant`**`(vMemAccM_VariantIdType VariantId)`	
Parameter	
`VariantId` [in]	Variant identifier to be set as the active variant.
Return code	
Std_ReturnType	E_OK Variant was set internally; E_NOT_OK otherwise
Functional Description	
Sets the active memory mapping variant to the passed value. The setting is synchronous, that means after the function returns, the variant is set and all following requests to an address area using the variant feature, the new variant will be used to forward the request to underlying module(s). See chapter 3.6.1 for more information about the memory mapping variants.	
Particularities and Limitations	
-	
Call context	
> TASK > This function is synchronous. > This function is reentrant.	

Table 5-19 vMemAccM_SetActiveVariant

5.2.17 vMemAccM_ErrorNotification

Prototype	
`Std_ReturnType` **`vMemAccM_ErrorNotification`**`(vMemAccM_AddressAreaIdType AddressAreaId, vMemAccM_AddressType Address, vMemAccM_ErrorNotifResultType Result)`	
Parameter	
AddressAreaId [in]	Numeric identifier of address area.
Address [in]	Virtual address causing the user to invoke the function.
Result [in]	Result to be combined with the actual vMem job result.
Return code	
Std_ReturnType	E_OK Result set internally; E_NOT_OK otherwise.
Functional Description	
Stores the passed result and combines it with the actual vMem job result, in case the passed AddressAreaId and the Address matches the currently active job. All other requests will be rejected. See chapter 3.5 for more information about the usage of this API.	

Particularities and Limitations
-
Call context
> ANY > This function is synchronous. > This function is reentrant.

Table 5-20 vMemAccM_ErrorNotification

5.2.18 vMemAccM_GetVersionInfo

Prototype	
`void vMemAccM_GetVersionInfo (Std_VersionInfoType *VersionInfo)`	
Parameter	
VersionInfo [out]	Pointer to where to store the version information. Parameter must not be NULL.
Return code	
void	none
Functional Description	
Returns the version information. `vMemAccM_GetVersionInfo()` returns version information, vendor ID and AUTOSAR module ID of the component.	
Particularities and Limitations	
VMEMACCM_VERSION_INFO_API == STD_ON	
Call context	
> TASK > This function is synchronous. > This function is reentrant.	

Table 5-21 vMemAccM_GetVersionInfo

5.3 Services used by vMemAccM

In the following table services provided by other components, which are used by the vMemAccM are listed. For details about prototype and functionality refer to the documentation of the providing component.

Component	API
DET (Development Error Tracer)	`Det_ReportError`

vMem	`vMem_Read`
	`vMem_Write`
	`vMem_Erase`
	`vMem_IsBlank`
	`vMem_GetJobResult`
	`vMem_<HwSpecificReq>`

Component	API
SchM (Schedule Manager)	SchM_Enter_vMemAccM_VMEMACCM_EXCLUSIVE_AREA_0 SchM_Exit_vMemAccM_VMEMACCM_EXCLUSIVE_AREA_0

Table 5-22 Services used by the vMemAccM

5.4 Callback Functions

The vMemAccM does not implement any callback functions.

5.5 Configurable Interfaces

5.5.1 Notifications

The vMemAccM does not define any notifications.

5.5.2 Callout Functions

The vMemAccM does not define any callout functions.

6 Configuration

In the vMemAccM the attributes can be configured according to/ with the following methods/ tools:

- DaVinci Configurator 5 (recommended)
- AUTOSAR Generic Configuration Editor (GCE)

6.1 Configuration Variants

The vMemAccM supports the configuration variants

- `VARIANT-PRE-COMPILE`

6.2 Configuration with DaVinci Configurator

The configuration parameters are generally well described within the tool and the vMemAccM_bswmd.arxml file. However, there are some more complex dependencies between certain configuration parameters which are described in more detail in the following.

6.2.1 Read Request Length

The maximum number of bytes the vMemAccM requests within one vMem read request can be configured for each address area within the parameter `vMemAccMReadRequestLength`.

If `vMemAccMCompareApi` is enabled, the configured values will also be used to calculate the internal buffer size – see chapter 3.11. Therefore, it is recommended to choose the read request length with care, since it determines the internal RAM usage.

6.2.2 Memory Address Mapping

To achieve a correct mapping from virtual to physical addresses the sub address areas must be configured according to the following instructions:

6.2.2.1 Physical Memory

- The sector batch where the physical memory of the related sub address area is located must be referenced in the parameter **Sector Ref**. All memory device specific information, such as page and sector size, are to be configured within the vMem in the sector batch.
- The physical memory of any sub address area must lie within the range of the referenced sector batch. A validator checks this for every sub address area. The following explanations shall only help to understand what the validation rule checks. The range of a sector batch is determined by its start and end address. The start address of the sector batch has to be configured in the vMem parameter **Start Address**. The end address of the sector batch can be computed using the vMem parameter **Number Of Sectors** and **Sector Size** using the formula

EndAddress = StartAddress + NumberOfSectors * SectorSize – 1

The physical start address of a sub address area is configured within the vMemAccM parameter **Physical Start Address**. The physical end address of the sub address area can be computed in the same way as the end address of the sector batch, but with the vMemAccM parameter **Number Of Sectors**. The validation rule checks that both the physical start address and physical end address lie within the range of the referenced sector batch.

> The same sector batch can be referenced from multiple sub address areas from different address areas. The physical memory of the sub address areas referencing the same sector batch must NOT overlap within this sector batch. This is also checked by a validator.

> The physical start address of any sub address area must be aligned. That is, it must be a multiple of the sector size of the referenced sector batch. This is also checked by a validator.

6.2.2.2 Virtual Memory

> The sub address areas must not overlap within the address area. A validator checks this for every address area. The following explanations shall only help to understand what the validation rule checks. The virtual start address of a sub address area is configured in the parameter **Virtual Start Address**. The virtual end address of a sub address area can be calculated using the formula

VirtualEndAddress = VirtualStartAddress + NumberOfSectors * SectorSize – 1

where NumberOfSectors is the number of sectors of the sub address. The validator checks, that for any pair of consecutive sub address areas, the virtual end address of the first sub address area is smaller than the virtual start address of the second sub address area.

> The virtual end address of a sub address area must not exceed the value 4294956295 or 0xFFFF FFFF in hexadecimal, since the type used within the implementation is uint32. This is also checked by a validator.

7 Glossary and Abbreviations

7.1 Glossary

Term	Description
Address area	Virtual area of memory, not necessarily continuously filled. Asynchronous jobs of vMemAccM are requested for an address area.
Page	Physical area of memory, smallest writable unit.
Page Burst / Write Burst	A page burst can be used for improved performance of write jobs. Instead of writing one page at a time, multiple pages within the same sector batch can be written at once.
Pseudo-physical address	The pseudo-physical addresses refer to the addresses which are provided by the memory driver. They are not the physical addresses used by the microprocessor.
Sector	Physical area of memory, smallest erasable unit.
Sector Burst / Erase Burst	A sector burst can be used for improved performance of erase jobs. Instead of erasing one sector at a time, multiple sectors within the same sector batch can be erased at once.
Sub address area	Part of an address area, which corresponds to a physical continuous area of memory.

Table 7-1 Glossary

7.2 Abbreviations

Abbreviation	Description
API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
BSW	Basis Software
DET	Development Error Tracer
ECU	Electronic Control Unit
EcuM	ECU Manager
Fee	Flash EEPROM Emulation
HSM	Hardware Security Module
ISR	Interrupt Service Routine
MemIf	Memory Abstraction Interface
MICROSAR	Microcontroller Open System Architecture (the Vector AUTOSAR solution)
NV, NVRAM	Non Volatile Random Access Memory
NvM	NVRAM Manager
OTA	Over The Air

Fls_30_vMemAccM	Adapter between AUTOSAR memory stack (Fee) and Software Download stack (vMemAccM) Note: previously this component was called vFlsAcc
vMem	Vector Memory (Driver)
vMemAccM	Vector Memory Access Manager
vSwUpdM	Vector Software Update Manager

Table 7-2 Abbreviations

8 Contact

Visit our website for more information on

- News
- Products
- Demo software
- Support
- Training data
- Addresses

www.vector.com